EECS151 ASIC Project Report

Alexander Volkov

May 8, 2024

Problem 1: Team Information

Please list the full name of the individuals. If you worked only, then write your name down as *Project Partner 1* and leave *Project Partner 2* blank.

Project Partner 1: Alexander Volkov

Project Partner 2: _____

Problem 2: Design

2.1 Summary:

Core Functional? (Yes/No): Yes

Number of Pipeline Stages: 4

Forwarding Implemented? (Yes/No): Yes, fully-bypassed

Branch Prediction Implemented? (Yes/No): No (trivial predict-not-taken)

Cache Implemented? (Yes/No): Yes

Cache Functional? (Yes/No): Yes

Cache Size: 4 KiB + 256 B metadata

Cache Associativity: Direct-mapped

Cache Read Miss Latency: Depends on if the line is valid/not-valid, dirty, Memory readiness. Best-case scenario: not-valid line, Memory is ready: 6 cycles. Worst-case scenario: Depends on Memory readiness, but add at least 4 cycles for write-back.

Cache Read Hit Latency: 1 cycle to check for tag, pipeline is continued on the next cycle

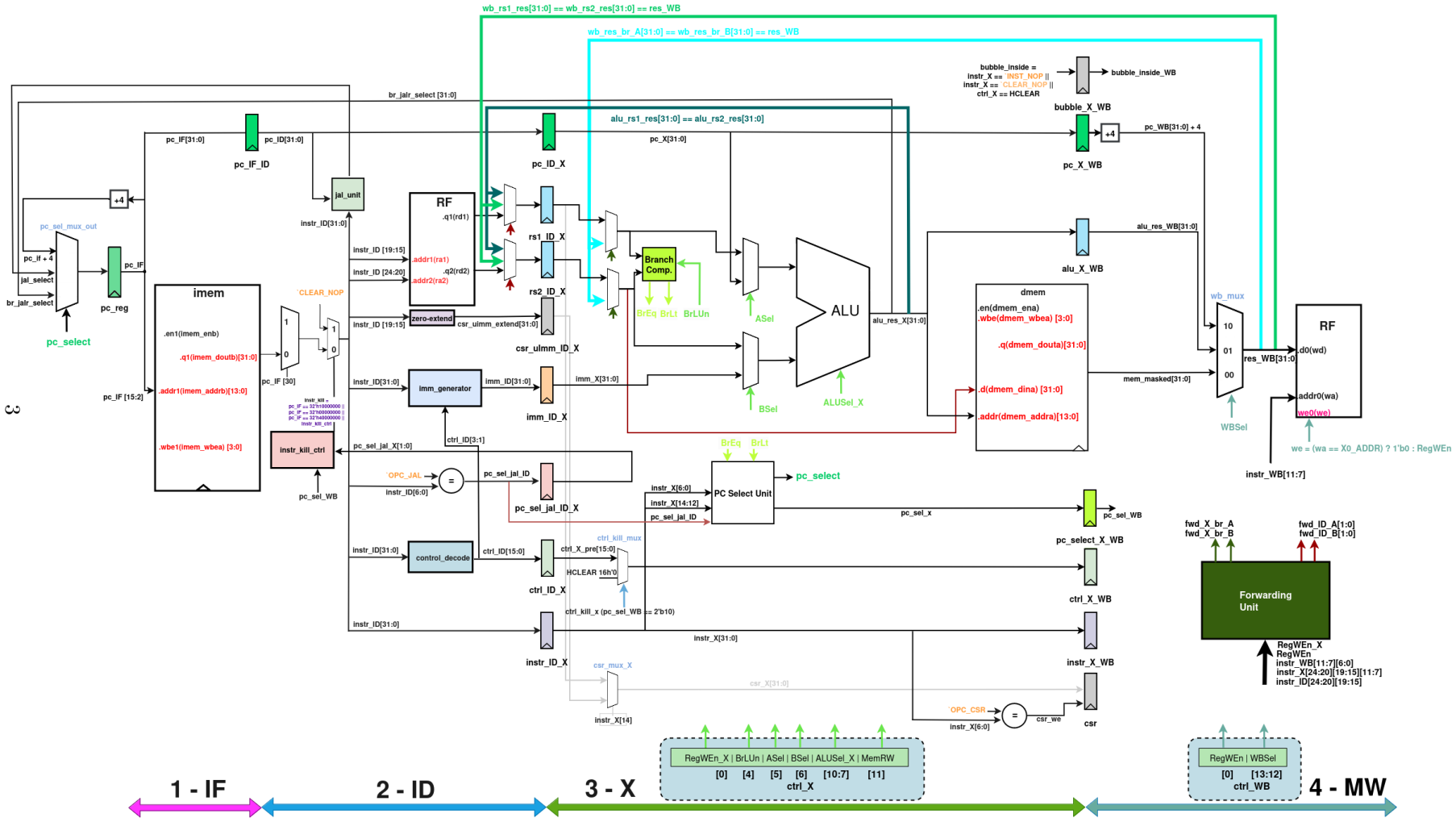
Cache Write Miss Latency: 7 cycles best-case, +4 extra cycles for write-back (if Memory is ready) for worst-case.

Cache Write Hit Latency: 1 cycle

Other Optimizations (Yes/No; please describe in later section): Yes, if JAL being calculated in the ID-stage (as opposed to the 3rd stage, which is EX) and PC immediately jumping to target, can be considered an optimization

2.2 Pipeline Design

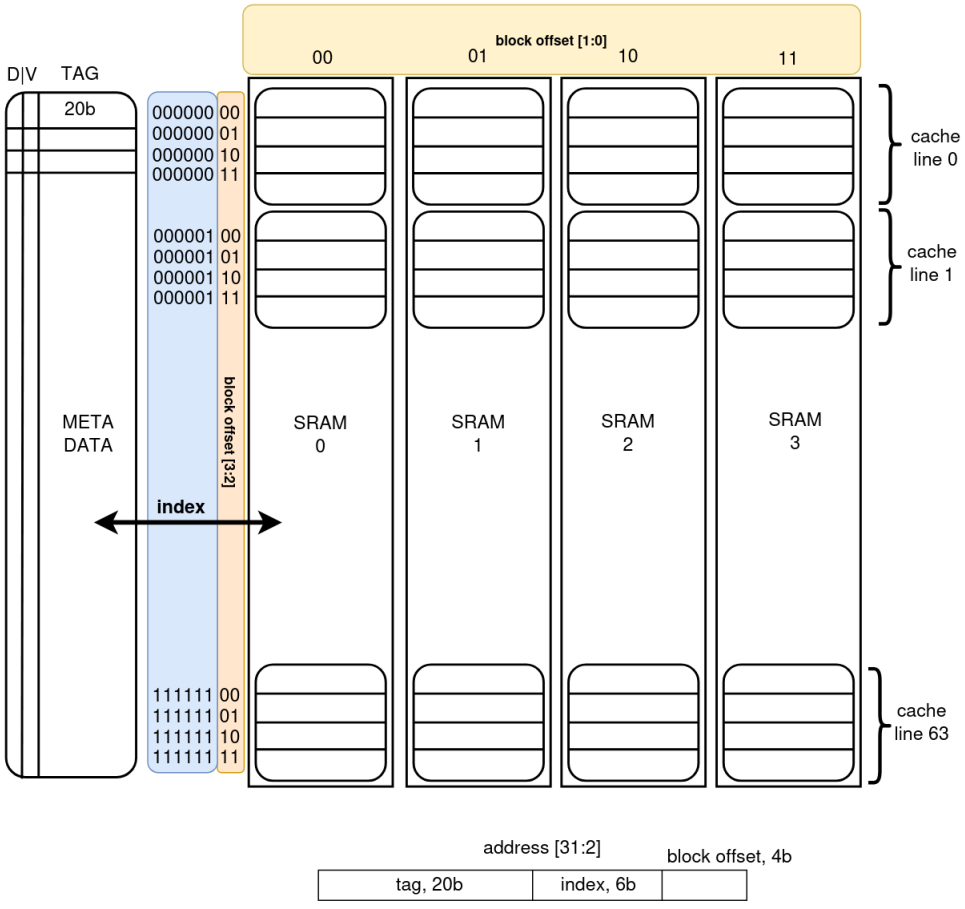
Pipeline diagram:


wb_rs1_res[31:0] == wb_rs2_res[31:0] == res_WB
wb_res_br_A[31:0] == wb_res_br_B[31:0] == res_WB
alu_rs1_res[31:0] == alu_rs2_res[31:0]
br_jalr_select [31:0]
bubble_inside = instr_X == `INST_NOP || instr_X == `CLEAR_NOP || ctrl_X == HCLEAR
bubble_inside_WB
bubble_X_WB
pc_WB[31:0] + 4
pc_X_WB
pc_IF[31:0]
pc_ID[31:0]
pc_IF_ID
pc_ID_X
pc_X[31:0]
+4
jal_unit
instr_ID[31:0]
RF
.q1(rd1)
.q2(rd2)
.addr1(ra1)
.addr2(ra2)
rs1_ID_X
rs2_ID_X
instr_ID [19:15]
instr_ID [24:20]
pc_sel_mux_out
pc_if + 4
jal_select
br_jalr_select
pc_select
pc_IF
pc_reg
imem
.en1(imem_enb)
.q1(imem_doutb)[31:0]
.addr1(imem_addrb)[13:0]
.wbe1(imem_wbea) [3:0]
pc_IF [30]
pc_IF [15:2]
`CLEAR_NOP
instr_ID [19:15]
zero-extend
csr_uimm_extend[31:0]
csr_uImm_ID_X
instr_ID[31:0]
imm_generator
imm_ID[31:0]
imm_X[31:0]
imm_ID_X
ctrl_ID[3:1]
instr_kill_ctrl
pc_sel_WB
pc_sel_jal_X[1:0]
`OPC_JAL
instr_ID[6:0]
pc_sel_jal_ID
pc_sel_jal_ID_X
Branch Comp.
BrEq BrLt BrLUn
ASel
BSel
ALU
ALUSel_X
alu_res_X[31:0]
alu_X_WB
alu_res_WB[31:0]
dmem
.en(dmem_ena)
.wbe(dmem_wbea) [3:0]
.q(dmem_douta)[31:0]
.d(dmem_dina) [31:0]
.addr(dmem_addra)[13:0]
mem_masked[31:0]
wb_mux
10
01
00
WBSel
RF
.d0(wd)
.addr0(wa)
we0(we)
res_WB[31:0]
instr_WB[11:7]
we = (wa == X0_ADDR) ? 1'b0 : RegWEn
BrEq BrLt
PC Select Unit
instr_X[6:0]
instr_X[14:12]
pc_sel_jal_ID
pc_select
pc_sel_x
pc_sel_WB
pc_select_X_WB
ctrl_kill_mux
control_decode
instr_ID[31:0]
ctrl_ID[15:0]
ctrl_X_pre[15:0]
HCLEAR 16'h0
ctrl_ID_X
ctrl_X_WB
ctrl_kill_x (pc_sel_WB == 2'b10)
instr_X[31:0]
instr_ID_X
instr_X_WB
csr_mux_X
csr_X[31:0]
instr_X[14]
`OPC_CSR
instr_X[6:0]
csr_we
csr
fwd_X_br_A
fwd_X_br_B
fwd_ID_A[1:0]
fwd_ID_B[1:0]
Forwarding Unit
RegWEn_X
RegWEn
instr_WB[11:7][6:0]
instr_X[24:20][19:15][11:7]
instr_ID[24:20][19:15]
RegWEn_X | BrLUn | ASel | BSel | ALUSel_X | MemRW
[0] [4] [5] [6] [10:7] [11]
ctrl_X
RegWEn | WBSel
[0] [13:12]
ctrl_WB
1 - IF
2 - ID
3 - X
4 - MW

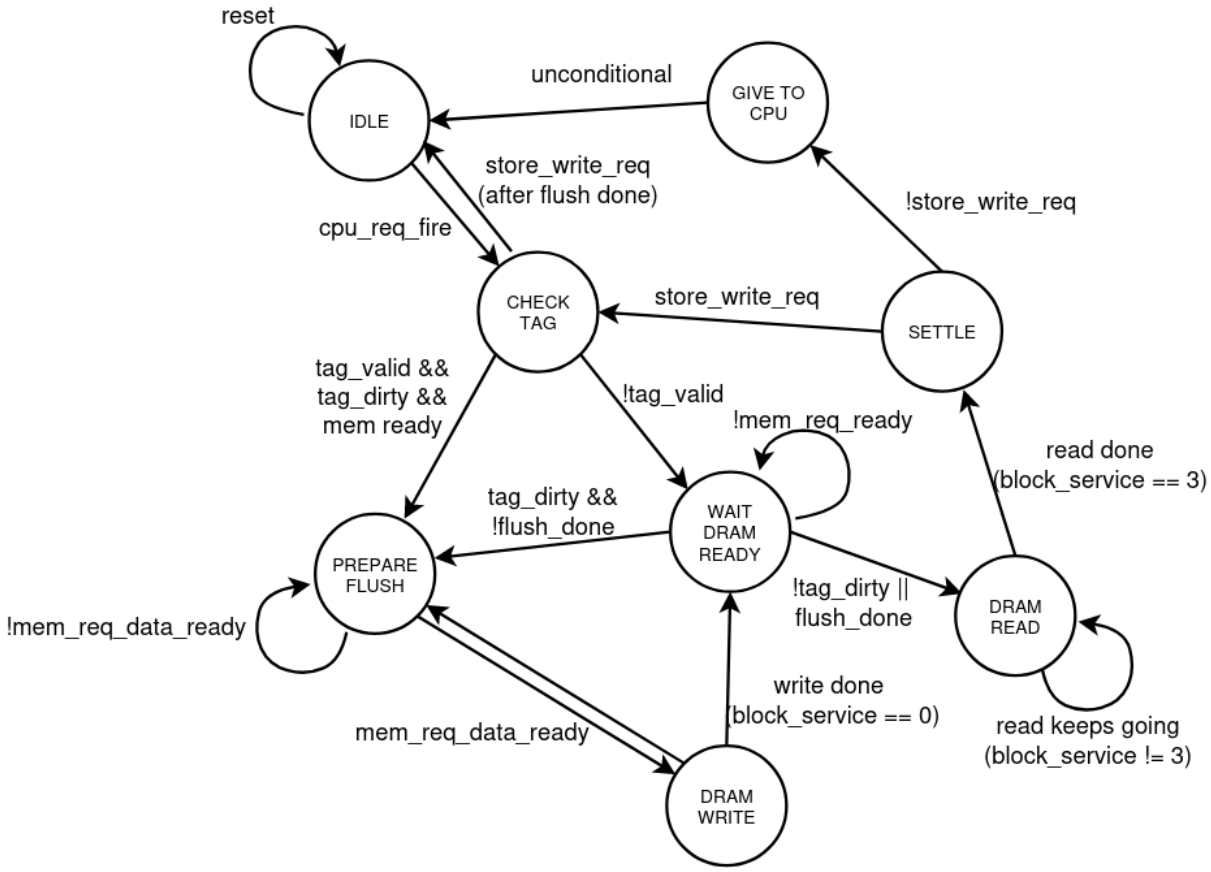
This is a fully-bypassed four-stage pipeline design with the original intent to keep the 1st stage utilized by introducing the branch-target buffer to efficiently deal with branches in some cases. The branch-prediction module is not bug-proof yet, and, hence, will not be included in the final submission. This was supposed to happen in the first stage. Moving onto the second stage – ID – the fetched instuction gets indentified, control signals get decoded, immediate gets generated, and the target registers are known. This stage also detects a JAL instruction, singals to PC-select block, kills the instruction that follows it immediately after, introduces a bubble instead of it, and lets the PC jump the target address. The third stage – EX – contains the ALU, Branch comparator, forwarding muxes (the previous stage also has them), and the PC-select unit (although, this unit spans both ID and EX stages, as it collects information from both in order to correctly set the next PC). The CSR instruction is detected at this stage as well. Moving onto the last stage – MEM/WB – it contains the DCache, as well as the WB-select mux. There is a forwarding logic calculator that effectively spans three stages – ID, EX, and MEM/WB in order to properly address forwarding. This design completely eliminates the need for a stall due to (in-order) dependencies (although, of course, the stalls do happen due to cache tag-checks at the very least). The load dependency is forwarded to the EX stage (in the worst-case scenario), and the other dependencies are forwarded to the ID stage.

2.3 Cache Design

Cache Organization:



Cache FSM:



The cache consist of four banks of 1 KiB each (with the individual SRAM size/dimensions are 256x32b), as well as the single 64x32b metadata bank. The logical element of the cache – the 16-word line – corresponds to the 4 lines for each of the physical SRAM block (see the diagram). In order to map an individual word to its particular place in the SRAM ensemble, the two LSBs of the block offset are used to select the physical SRAM, and the other two bits are used to select the line in the 4-word sub-block of the chosen SRAM by taking the entire 8-bit section (the actual index + 2 MSBs of the block offset). The metadata SRAM uses the 6-bit index only, because its line count is actually 64, and thus, no other bit selection is necessary.

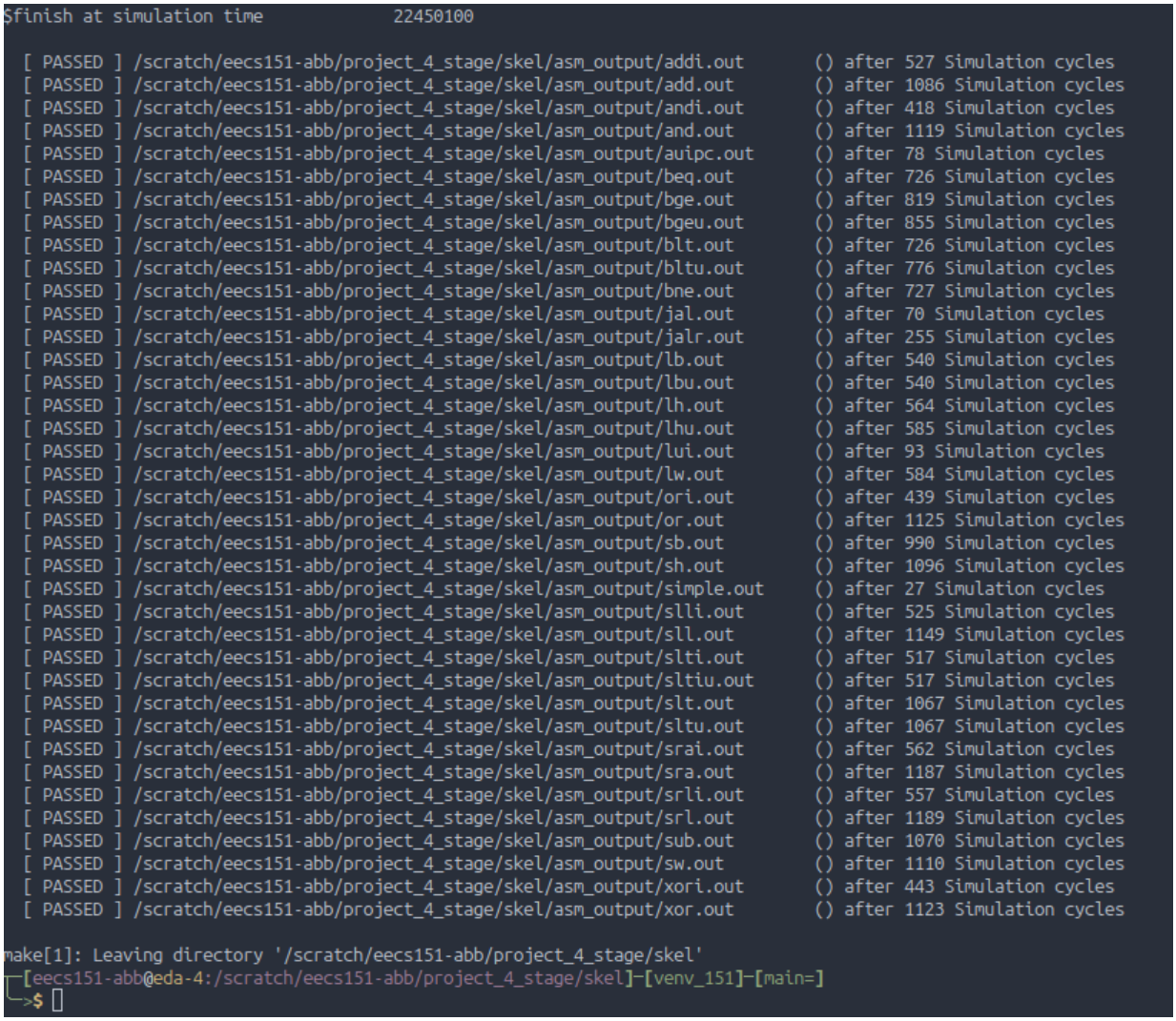
There are eigth states in the cache controller FSM, and a few hold-registers that hold the passed address/data. The states are named as follows: IDLE, CHECK_TAG, WAIT_DRAM_READY, DRAM_READ, SETTLE, GIVE_TO_CPU, PREPARE_FLUSH, and DRAM_WRITE. The cache starts in the IDLE state. As soon as the request from the cpu signals that instruction/data is needed the cache transitions into the CHECK_TAG state. In this state, a few things get checked – whether the tag is valid (which, if it is not, the cache will transition into the WAIT_DRAM_READY state on the next cycle), whether the tags match, and whether the request is read or write. If the tags don't match, the dirty flag is checked. If the tag is not dirty, or if it is dirty, but the memory isn't ready yet, the state will transition to WAIT_DRAM_READY as well. Otherwise, the next state is PREPARE_FLUSH. In the latter state, the controller tells the memory that request is a write, gets the first four words from the data SRAMs, and depending on the memory readiness, either spins in this state, or transisitions to DRAM_WRITE where it spends additional 3 cycles in order write-back all of the words from the give line; after which it will have to wait for the dram to be ready, and if it is – immediately starts reading. The read takes 4 cycles, and if the request was a write, the state will transition to the CHECK_TAG state where it will just write the data to the physical SRAM and release

the pipeline. If the request was a read, the next state will give the cpu the actual data and also release the pipeline hold.

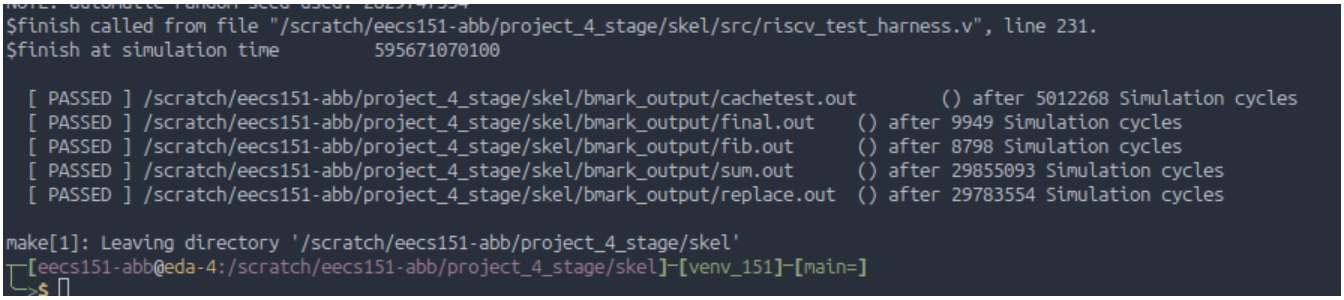
Problem 3: Simulation

3.1 Behavioral

Did you pass all the assembly tests (*asm* tests)? Yes



Did you pass all the benchmark tests (*bmark* tests)? Yes



List the reported cycles for each test (both assembly and benchmark). Leave the corresponding cell blank if you failed a test.

addi	527	*lb*	540	*slti*	517
add	1086	*lbu*	540	*sltiu*	517
andi	418	*lh*	564	*slt*	1067
and	1119	*lhu*	585	*sltu*	1067
auipc	78	*lui*	93	*srai*	562
beq	726	*lw*	584	*sra*	1187
bge	819	*ori*	439	*srli*	557
bgeu	855	*or*	1125	*srl*	1189
blt	726	*sb*	990	*sub*	1070
bltu	776	*sh*	1096	*sw*	1110
bne	727	*simple*	27	*xori*	443
jal	70	*slli*	525	*xor*	1123
jalr	255	*sll*	1149		

Table 1: Reported cycles for assembly tests

cachetest	5012268
final	9949
fib	8798
sum	29855093
replace	29783554

Table 2: Reported cycles for benchmark tests

3.2 Post-Synthesis DO NOT FILL IN FOR SPRING 2024

Did you pass all the assembly tests (*asm* tests)? If not, please list the tests you failed and why (if known). Include a screenshot of the passing ASM tests.

Did you pass all the benchmark tests (*bmark* tests)? If not, please list the tests you failed and why (if known). Include a screenshot of the passing bmark tests.

List the reported cycles for each test (both assembly and benchmark). Leave the corresponding cell blank if you failed a test.

addi		*lb*		*slti*	
add		*lbu*		*sltiu*	
andi		*lh*		*slt*	
and		*lhu*		*sltu*	
auipc		*lui*		*srai*	
beq		*lw*		*sra*	
bge		*ori*		*srli*	
bgeu		*or*		*srl*	
blt		*sb*		*sub*	
bltu		*sh*		*sw*	
bne		*simple*		*xori*	
jal		*slli*		*xor*	
jalr		*sll*			

Table 3: Reported cycles for assembly tests

cachetest	
final	
fib	
sum	
replace	

Table 4: Reported cycles for benchmark tests

Problem 4: Implementation: Synthesis

4.1 Timing

Does your design pass timing? (Yes/No): Yes

Maximum frequency for which the design passes timing (MHz): 125MHz, 8ns (zero slack) and 111MHz, 9ns (positive slack)

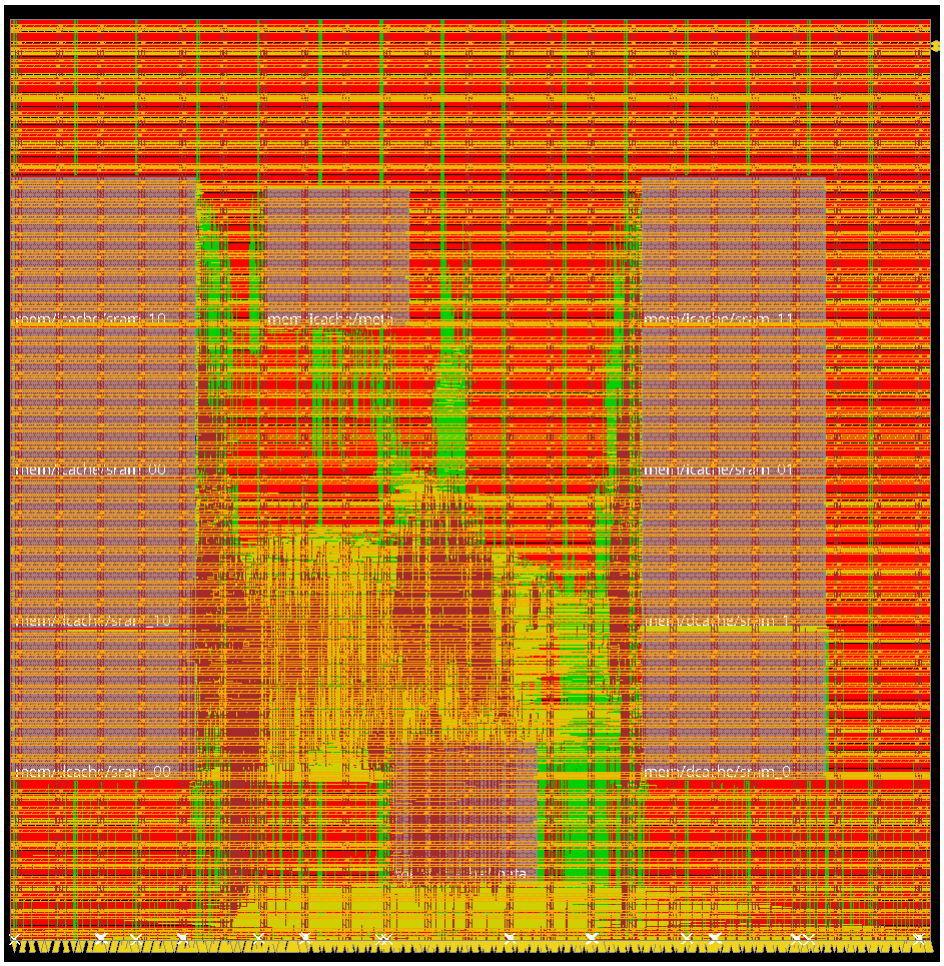
Slack on Critical Path (ps): 0ns for 8ns, and 254ns for 9ns

Critical Path Location (can list start and end point): ALU_X_WB – DCache store_write_req_reg

The critical path begins at the EX-MEM/WB pipeline register (the result of the ALU), takes on the forwarding path back into and through the ALU, and, finally, enters the DCache controller logic stopping at one of the internal hold registers. This is the most intensive path (in the sense of the number of gates involved) in the design. The ALU takes up a significant part of it, and, in fact, the path stays pretty much the same (with the slight variations of the path taken withing the ALU itself) post-par as well.

Problem 5: Implementation: Place-and-Route

5.1 Floorplan



The floorplan strategy is a combination of both the theoretical (rules of thumb, first principles) and the empirical strategies. The main SRAM banks are located on the sides (with the right hand side shifted a little closer to the center. This "trick" helps increase the slack somewhat, as opposed to the original, very symmetric placement. The main SRAM banks are on metal3 with pins looking inward. The core cpu and memory logic are in the middle with the placement optimized away by Innovus free of any soft-macro directives. The most curious part is the positioning of the metadata SRAMs. Both, the ICache and the DCache ones play their own curious role in the floorplan organization when maximizing the slack: it is essentially the "empirical" part, the trial-and-error methodology, in action. Here's what has been observed: the ICache metadata SRAM doesn't affect timing at all, it can be placed pretty much anywhere in a reasonable spot with only one caveat – it shouldn't be located too close to its DCache counterpart. The placement of the latter, on the other hand, is critical to be able to meet timing. The best location that maximizes the slack is found to be almost in between of the main SRAM column banks, shifted west a little bit, pins facing north, and located on M4 layer. This somewhat unusual combination of the positions of the SRAMs has given the best timing results.

5.2 Timing

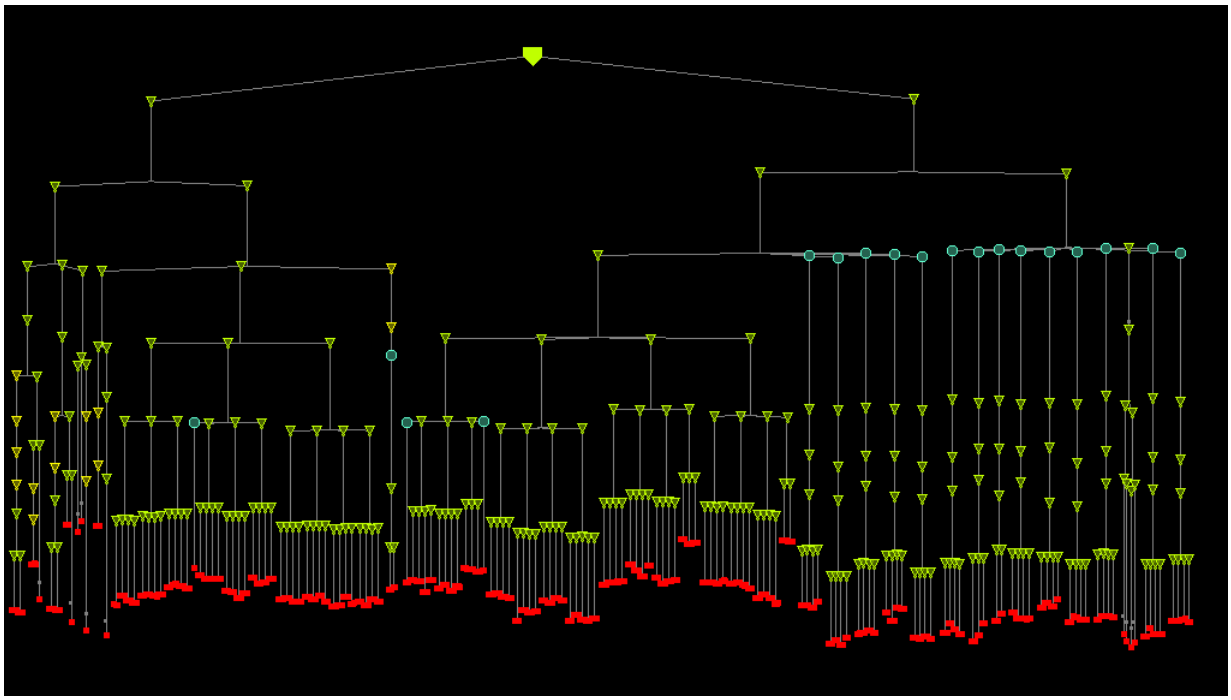
Does your design pass timing? (Yes/No): Yes

Maximum frequency for which the design passes timing (MHz): 55.56 MHz, 18ns

Slack on Critical Path (ps): 0.055ns

Critical Path Location (can list start and end point):

From: ALU_X_WB (a pipeline register between EX and MEM/WB)
To: Dcache address (clock gating logic, presumably for optimizing for power)



The clock tree looks balanced (the timing is met), but the stucture is somewhat irregular (probaby an indicator that some microarchitectural optimizations might be beneficial for increasing the clock period further) – the clock gating on the right-hand side is mostly due to pipeline registers (for stalling the pipeline during a cache request).

Setup slack:

```
##------------------------------------------------------------
Path 1: MET (0.055 ns) Clock Gating Setup Check with Pin mem/dcache/CLKGATE_RC_CG_HIER_INST14/RC_CGIC_INST/CLK->GATE
              View: ss_100C_1v60.setup_view
             Group: reg2cgate
        Startpoint: (R) cpu/alu_X_WB/q_reg[0]/CLK
             Clock: (R) clk
          Endpoint: (F) mem/dcache/CLKGATE_RC_CG_HIER_INST14/RC_CGIC_INST/GATE
             Clock: (R) clk

                     Capture       Launch
        Clock Edge:+   18.000        0.000
       Src Latency:+   -1.294       -1.294
       Net Latency:+    0.482 (P)    1.375 (P)
           Arrival:=   17.187        0.081

 Clock Gating Setup:-    0.493
          Uncertainty:-    0.100
          Cppr Adjust:+    0.000
        Required Time:=   16.595
         Launch Clock:=    0.081
            Data Path:+   16.459
                Slack:=    0.055
       Timing Path:

#------------------------------------------------------------
```

Hold slack:

```
##------------------------------------------------------------
Path 1: MET (0.099 ns) Hold Check with Pin cpu/csr_reg/q_reg[23]/CLK->D
              View: ff_n40C_1v95.hold_view
             Group: clk
        Startpoint: (R) reset
             Clock:
          Endpoint: (F) cpu/csr_reg/q_reg[23]/D
             Clock: (R) clk

                     Capture       Launch
        Clock Edge:+    0.000        0.000
       Src Latency:+   -0.611        0.000
       Net Latency:+    0.655 (P)    0.000 (I)
           Arrival:=    0.045        0.000

              Hold:+   -0.037
       Uncertainty:+    0.100
       Cppr Adjust:-    0.000
     Required Time:=    0.108
      Launch Clock:=    0.000
         Data Path:+    0.207
             Slack:=    0.099
      Timing Path:

#------------------------------------------------------------------------------------------------------------
# Timing Point            Flags  Arc    Edge  Cell                    Fanout  Trans   Delay  Arrival
#                                                                             (ns)    (ns)     (ns)
#------------------------------------------------------------------------------------------------------------
  reset                   -      reset  R     (arrival)                   45  0.003   0.193    0.193
  reset                   -      -      R     (net)                       45      -       -        -
  cpu/csr_reg/g241/Y      -      A->Y   F     sky130_fd_sc_hd__nor2b_1     1  0.515   0.013    0.207
  cpu/csr_reg/n_18        -      -      F     (net)                        1      -       -        -
  cpu/csr_reg/q_reg[23]/D -      D      F     sky130_fd_sc_hd__dfxtp_1     1  0.025   0.000    0.207
#------------------------------------------------------------------------------------------------------------
```

Roughly the same critical path shows up regardless of the stage (syn/par) and of the clock period set. It is

the forwarding path back to the ALU from the MEM/WB stage. The two small variations on the critical path are: 1) the functional unit within the ALU – sometimes it is the adder/subtractor, other times it is the shifter – but the overall general path trend doesn't change much – , and 2) The endpoint within the cache controller – sometimes is it the clock gate, sometimes it is the hold register for the fetched data).

5.3 Area

```
#@ End verbose source /scratch/eecs151-abb/bear_iv/skel/build/par-rundir/generated-scripts/open_chip.tcl
@innovus 2> report_area
Depth  Name                                                                  #Inst  Area (um^2)
--------------------------------------------------------------------------------------------
0      riscv_top                                                             10243  1093549.4602
1      mem                                                                   2939   1017800.561
1      cpu                                                                   7065   72202.9984
2      cpu/BranchComp                                                        137    929.6416
2      cpu/JALTarget                                                         141    927.1392
2      cpu/csr_mux_X                                                         32     277.7664
2      cpu/ctrl_ID_X                                                         30     362.848
2      cpu/instr_kill_mux                                                    33     222.7136
2      cpu/alu_A_mux                                                         47     376.6112
2      cpu/instr_ID_X                                                        59     740.7104
2      cpu/pc_reg                                                            74     938.4
2      cpu/alu_X_WB                                                          74     944.656
2      cpu/csr_uImm_ID_X                                                     13     161.4048
2      cpu/StoreMask                                                         64     477.9584
2      cpu/imm_ID_X                                                          74     930.8928
2      cpu/pc_select_X_WB                                                    3      33.7824
2      cpu/fw_ID_mux_rs1                                                     66     449.1808
2      cpu/CtrlDec                                                           26     160.1536
2      cpu/MemLoadMasked                                                     120    784.5024
2      cpu/csr_reg                                                           81     952.1632
2      cpu/wb_mux                                                            78     666.8896
2      cpu/fwd_br_mux_A                                                      45     371.6064
2      cpu/pc_ID_X                                                           74     932.144
2      mem/icache                                                            1299   507199.8997
2      mem/arbiter                                                           105    432.9152
2      cpu/ctrl_kill_mux                                                     12     80.0768
2      cpu/fw_ID_mux_rs2                                                     66     449.1808
2      cpu/pc_sel_jal_ID_X                                                   3      33.7824
2      cpu/InstrKill                                                         1      6.256
2      cpu/pc_X_WB                                                           74     932.144
2      cpu/rs2_ID_X                                                          74     930.8928
2      cpu/alu                                                               1118   8096.5152
2      cpu/ctrl_X_WB                                                         9      112.608
2      cpu/rf2                                                               3721   43200.1824
2      cpu/ImmGen                                                            49     361.5968
2      cpu/pc_IF_ID                                                          74     953.4144
2      cpu/alu_B_mux                                                         41     375.36
2      cpu/rs1_ID_X                                                          74     939.6512
2      cpu/PCSelect                                                          9      70.0672
2      cpu/fwd_br_mux_B                                                      39     394.128
2      cpu/instr_X_WB                                                        38     480.4608
2      mem/dcache                                                            1527   510077.6597
2      cpu/pc_sel_mux                                                        67     452.9344
2      cpu/forwarding                                                        62     451.6832
3      {cpu/rf2/genblk1[18].reg_x}                                           32     640.6144
3      {cpu/rf2/genblk1[14].reg_x}                                           32     640.6144
3      {cpu/rf2/genblk1[8].reg_x}                                            32     640.6144
3      {cpu/rf2/genblk1[5].reg_x}                                            32     640.6144
3      cpu/rs2_ID_X/CLKGATE_RC_CG_HIER_INST13                                1      18.768
3      {cpu/rf2/genblk1[31].reg_x}                                           32     640.6144
3      cpu/MemLoadMasked/h_mux                                               18     160.1536
3      {cpu/rf2/genblk1[25].reg_x}                                           32     640.6144
3      {cpu/rf2/genblk1[22].reg_x}                                           32     640.6144
3      cpu/ctrl_X_WB/CLKGATE_RC_CG_HIER_INST4                                1      18.768
3      cpu/pc_reg/CLKGATE_RC_CG_HIER_INST11                                  1      18.768
3      {cpu/rf2/genblk1[4].reg_x}                                            32     640.6144
3      mem/dcache/hold_instr_reg                                             74     933.3952
3      {cpu/rf2/genblk1[0].reg_x}                                            32     640.6144
3      {cpu/rf2/genblk1[12].reg_x}                                           32     640.6144
3      cpu/ImmGen/ImmGenMux                                                  41     310.2976
3      cpu/pc_IF_ID/CLKGATE_RC_CG_HIER_INST9                                 1      20.0192
3      cpu/rs1_ID_X/CLKGATE_RC_CG_HIER_INST12                                1      20.0192
3      {cpu/rf2/genblk1[29].reg_x}                                           32     640.6144
3      {cpu/rf2/genblk1[23].reg_x}                                           32     640.6144
3      cpu/instr_X_WB/CLKGATE_RC_CG_HIER_INST7                               1      20.0192
3      {cpu/rf2/genblk1[2].reg_x}                                            32     640.6144
```

The largest area is consumed by the memory module. There are ten total physical SRAM hard macros, each SRAM cell is 6 transistors, plus decoders, sense amps and some other logic structures. The core pipeline,

on the other hand, contains relatively few registers, and mostly combinatorial elements.

Problem 6: Known Bugs

No bugs in the pipeline, nor in the cache were observed.

Problem 7: Optimizations (Optional)

There are two performance optimizations: Early unconditional branch target passing, and full bypassing.

If you would like to be included in the design competition, include your performance-area score for the *sum* benchmark:

$$Score = ClockPeriod * Cycles * Area^{\frac{1}{2}}$$

The units for *ClockPeriod* and *Area* should be ns and μm. Please refer to Checkpoint 4 for more specifics and an example. List the values for each variable.

ClockPeriod (ns)	18
Cycles	29855093
Area (μm)	72202.9984

Score for *sum* benchmark:

$$18 \cdot 10^{-9} \cdot 29855093 \cdot (72202.9984)^{1/2} = 144.4$$

Problem 8: Extra Information

All good. Branch prediction didn't get finished, but oh well :)

Problem 9: Feedback

Everything's good! The TAs are great. The labs are cool.